

교재 : 이것이 취업을 위한 코딩테스트다 with 파이썬 <https://www.youtube.com/watch?v=7C9RgOcvkvo&list=PLRx0vPvIEmdAghTr5mXQxGpHjWqSz0dgC&index=3>

시간복잡도

파이썬은 1억 정도의 연산을 하는데 1초가 걸린다고 생각하기

그리디(Greedy)

매 순간 최적의 선택을 함으로써 최적의 결과를 도출해내는 알고리즘

<그리디 문제 판별법>

1. 한 선택이 다른 선택에 영향을 주지 않으면서, 최선의 선택이 무엇인지 명확히 알 수 있는가? -> 현재의 선택이 다음 선택에 영향을 준다면 어떤 선택이 최선인지 알기 힘들다. 당시에는 최선이었던 선택을 나중에 바꾸어야 하는 경우, 그리디한 문제라고 할 수 없다.
2. 최선의 선택을 반복하여 정답을 도출할 수 있는가? -> 그리디 알고리즘을 통해 최적의 해가 보장되는지 정당성 분석

ex) 거스름 돈 문제 BOJ#5585 정당성 분석 : 큰 단위가 항상 작은 단위의 배수이므로

```
n = int(input())
arr = list(map(int, input().split()))
stack = [(0,0)]
ans = []
for i in range(n):
    while stack:
        tower,idx = stack.pop()
        if tower > arr[i]:
            ans.append(idx+1)
            stack.append((tower,idx))
            break
        else:
            continue
    else:
        ans.append(0)
        stack.append((arr[i],i))
print(*ans)
```

스택

파이썬에서는 스택을 리스트를 통해 구현할 수 있다.

```
stack = []
stack.append(5) # 제일 끝에 5 삽입
stack.pop() # 제일 끝 원소 삭제
```

```
stack.pop(-1) # 제일 앞 원소 삭제
stack.peek()
top = stack[-1] # 제일 끝 원소 저장
stack.empty()

print(stack[::-1]) # 꺼낼 순서대로 출력
```

동일한 c++코드는 다음과 같다.

```
stack<int> s;

int main(void){
    s.push(5);
    s.pop();

    while(!s.empty()) {
        cout << s.top() << ' ';
        s.pop();
    }
}
```

큐

파이썬의 리스트로 구현 가능하나, 시간복잡도 때문에 deque라이브러리를 사용하는 것이 좋다. ->스택과 큐 자료구조의 장점을 합친 형태

```
from collections import deque

q = deque()

q.append(5)
q.popleft()
q.reverse() # 나중에 들어온 원소부터 출력
print(q)
```

재귀함수 - 최대공약수 계산(유클리드 호제법)

유클리드 호제법: 자연수 A를 B로 나눈 나머지를 R이라고 할 때, A와 B의 최대공약수는 B와 R의 최대공약수와 같다. 이를 R이 0이 될 때까지 반복하면, R이 0일 때의 B가 최대공약수가 된다.

```
def gcd(a,b):
    if a%b == 0:
        return b
```

```
else:
    return gcd(b, a%b)
```

그래프

1. 인접행렬 정점 ij 가 연결되어 있다면 $graph[i][j]$ 의 값을 1로. (양방향 그래프의 경우 인접행렬이 대칭)

특정 i, j 가 연결되어 있는지 확인 $O(1)$ 특정 정점과 연결된 모든 정점 확인 $O(V)$ 공간복잡도 $O(V^2)$

2. 인접리스트 V 개의 연결리스트 각 리스트에 연결된 정점들의 정보 담음.

특정 ij 가 연결되어 있는지 확인 $O(\min(\text{degree}(I), \text{degree}(J)))$ 특정 정점과 연결된 모든 정점 확인 $O(\text{degree}(X))$ 공간복잡도 $O(V+E)$

DFS

스택 또는 재귀함수를 이용

1. 시작 노드를 스택에 삽입하고 방문 처리
2. 스택의 최상단 노드에 방문하지 않은 인접한 노드가 하나라도 있으면 그 노드를 스택에 넣고 방문 처리. 방문하지 않은 인접 노드가 없으면 스택의 최상단 노드를 꺼낸다.
3. 더 이상 2번을 수행할 수 없을 때까지 반복

재귀함수 이용 - 인접리스트

```
n,m = list(map(int, input().split()))
adj=[[] for _ in range(n)] # 인접리스트
visited = [False] * n

for _ in range(m):
    u, v = list(map(int, input().split()))
    adj[u].append(v)
    adj[v].append(u) # 무방향 그래프인 경우

def dfs(x):
    visited[x]= True # 현재 노드를 방문 처리
    for i in adj[x]: # v번 노드와 인접한 노드 하나씩
        if not visited[i]:
            dfs(i)

dfs(1)
```

재귀 이용 - 2차원 평면 전체가 그래프인 경우

```

n,m = list(map(int, input().split()))
graph = [list(map(int,input().split())) for _ in range(m)]
visited = [[False]*n for _ in range(m)]
dx,dy = [1,0],[0,1]

def dfs(x,y):
    visited[x][y] = True
    for i in range(2):
        nx, ny = x + dx[i], y + dy[i]
        if 0<=nx<m and 0<=ny<n and not visited[nx][ny]:
            dfs(nx,ny)

dfs(0,0)

```

스택 이용 - 인접리스트

```

n,m = list(map(int, input().split()))
adj=[[] for _ in range(n)] # 인접리스트
visited = [False] * n

def dfs(x):
    stack = [x]
    while(stack): #스택에 남은것이 없을 때까지 반복
        node = stack.pop() # node : 현재 방문
        #현재 node가 방문한 적 없다 -> visited에 추가한다.
        #그리고 해당 node의 자식 node들을 stack에 추가한다.
        if not visited[node]:
            visited[node] = True
            stack.extend(adj[node])

```

BFS

큐 자료구조를 이용한다.

1. 시작 노드를 큐에 삽입하고 방문 처리
2. 큐에서 노드를 꺼낸 후, 해당 노드의 인접 노드 중 방문하지 않은 노드를 모두 큐에 삽입.
3. 더 이상 2번을 수행할 수 없을 때까지 반복

간선의 비용이 동일한 상황에서 최단거리 문제를 해결할 수 있다.

```

from collections import deque

def bfs(x):
    q = deque([x])
    visited[x] = True

    while q:
        v = q.popleft()
        for i in graph[v]:
            if not visited[i]:
                q.append(i)
                visited[i] = True

```

2차원 평면 전체가 그래프인 경우

```

from collections import deque
n,m = list(map(int, input().split()))
graph = [list(map(int,input().split())) for _ in range(m)]
visited=[[0]*n for _ in range(m)]
dx,dy = [1,0,-1,0],[0,-1,0,1]

def bfs(x,y):
    q = deque([x,y]) # 시작 노드 큐에 넣고
    visited[x][y] = True # 방문 표시

    while q: # 큐가 빌 때 까지
        x,y = q.popleft()

        for i in range(4):
            nx, ny = x+dx[i], y+dy[i]
            if 0<=nx<m and 0<=ny<n and not visited[nx][ny]:
                q.append([nx,ny])
                visited[nx][ny] = True

```

완전탐색(브루트포스)

: 발생할 수 있는 모든 경우를 탐색하는 방법이다.

- 선형구조 : 순차탐색
- 비선형구조 : 백트래킹, DFS, BFS

다음과 같은 방식이 존재한다.

1. 반복문/조건문을 통해 가능한 모든 경우를 찾는다.

2. 순열 :

3.

이진 탐색

: 정렬되어 있는 리스트에서 탐색 범위를 절반씩 좁혀가며 데이터를 탐색하는 방법 - $O(\log n)$

- 파이썬 이진탐색 라이브러리
 - `bisect_left(a, x)` --> 정렬된 순서를 유지하면서 리스트 a에 데이터 x를 삽입할 가장 왼쪽 인덱스를 찾는 메소드
 - `bisect_right(a, x)` --> 정렬된 순서를 유지하도록 리스트 a에 데이터 x를 삽입할 가장 오른쪽 인덱스를 찾는 메소드
- 이진탐색이 항상 빠른 것은 아님
 - 정렬되어 있는 경우
 - 정렬되어 있지 않은 경우 -
- 이진 탐색 소스코드 구현 (재귀 함수)

```
def binary_search(array, target, start, end):
    if start > end:
        return None
    mid = (start + end) // 2
    # 찾은 경우 중간점 인덱스 반환
    if array[mid] == target:
        return mid
    # 중간점의 값보다 찾고자 하는 값이 작은 경우 왼쪽 확인
    elif array[mid] > target:
        return binary_search(array, target, start, mid - 1)
    # 중간점의 값보다 찾고자 하는 값이 큰 경우 오른쪽 확인
    else:
        return binary_search(array, target, mid + 1, end)

# 이진 탐색 수행 결과 출력
result = binary_search(array, target, 0, n - 1)
print(result + 1)
```

- 이진 탐색 소스코드 구현 (반복문)

```
def binary_search(array, target, start, end):
    while start <= end:
```

```

        mid = (start + end) // 2

        if array[mid] == target:
            return mid
        elif array[mid] > target:
            end = mid - 1
        else:
            start = mid + 1
    return None

result = binary_search(array, target, 0, n - 1)
print(result + 1)

```

- 찾는 값이 리스트에 여러 개 있다면, **어떤 값이 return될 지 모른다!** -> 이 때는 lower bound부터 upper bound까지가 찾는 값들의 위치이며, 데이터가 배열에 없다면 lower bound == upper bound
- **lower bound** target 이상의 값이 최초로 나오는 위치

```

```python
def lower_bound(array, target, start, end):
 while start <= end:
 mid = (start + end) // 2

 if array[mid] >= target:
 end = mid - 1
 min_idx = min(min_idx, mid)
 else:
 start = mid + 1

 return min_idx
```

```

- **upper bound** target을 초과하는 값이 최초로 나오는 위치

```

```python
def upper_bound(array, target, start, end):
 while start <= end:
 mid = (start + end) // 2

 if array[mid] > target:
 end = mid - 1
 min_idx = min(min_idx, mid)
 else:
 start = mid + 1

 return min_idx

```

```

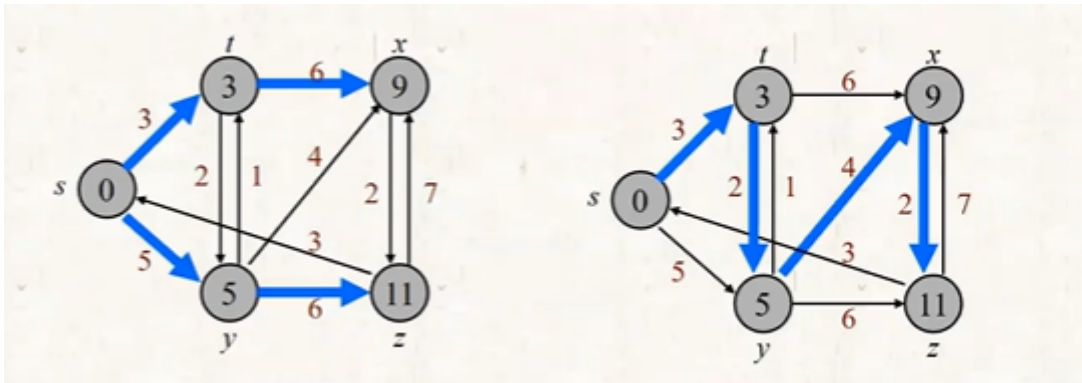
...
(lower bound 코드에서 등호 하나만 제외)
(lower bound, upper bound 모두 $O(\log N)$)
(upper bound는 항상 lower bound보다 크다.)

```

## 최단 경로 알고리즘

1. 한 지점 - 한 지점까지의 최단경로
2. 한 지점 - 다른 모든 지점까지의 최단경로 ex) 다익스트라 알고리즘
  - 다른 모든 문제를 푸는 데 이용될 수 있다.
  - 음의 값을 갖는 cycle이 없다면 음의 가중치를 갖는 그래프에서도 사용 가능(이 때, cycle을 포함하지 않는 경로가 항상 존재)
3. 모든 지점 - 한 지점까지의 최단경로
4. 모든 지점 - 다른 모든 지점까지의 최단경로 4-1.

**predecessor subgraph** 한 지점(s)에서 다른 모든 지점까지의 최단 경로를 보여주는 트리 (여러 개가 존재할



수 있다.)

각 노드는 predecessor만 저장하며,  $O(V)$ 의 공간복잡도를 가진다.

**relaxation**

### 2-1. 다익스트라 알고리즘

특정 노드로부터 다른 모든 노드로 가는 최단 거리를 계산

- 음의 간선이 없을 때 정상적으로 동작한다.
- 그리디 알고리즘으로 분류된다. (매 상황 가장 비용이 적은 노드 선택)
  1. 방문하지 않은 노드 중 최단 거리가 가장 짧은 노드 선택
  2. 해당 노드를 거쳐 다른 노드로 가는 비용을 모두 계산하고 원래 테이블 값과 비교해 더 작은 값으로 최단 거리 테이블 갱신할 때까지 이 과정을 반복한다.
- 한 번 방문처리된 노드의 최단거리는 고정되어 바뀌지 않는다.



아래 코드는 시간복잡도  $O(V^2)$ 으로, 전체 노드 갯수  $\leq 5000$ 인 경우에 사용한다.

```
import sys
input = sys.stdin.readline
INF = int(1e9) # 무한을 의미하는 값으로 10억을 설정

n, m = map(int, input().split())
graph = [[] for i in range(n + 1)]
start = int(input()) # 시작 노드 번호
visited = [False] * (n + 1)
distance = [INF] * (n + 1) # start 노드에서 n번 노드(n번 인덱스)까지의 최단거리

모든 간선 정보를 입력받기
for _ in range(m):
 a, b, c = map(int, input().split())
 # a번 노드에서 b번 노드로 가는 비용이 c라는 의미
 graph[a].append((b, c))

방문하지 않은 노드 중에서, 가장 최단 거리가 짧은 노드의 번호를 반환
def get_smallest_node():
 min_value = INF
 index = 0 # 가장 최단 거리가 짧은 노드(인덱스)
 for i in range(1, n + 1):
 if distance[i] < min_value and not visited[i]:
 min_value = distance[i]
 index = i
 return index

def dijkstra(start):
 # 시작 노드에 대해서 초기화
 distance[start] = 0
 visited[start] = True
 for j in graph[start]:
 distance[j[0]] = j[1]
 # 시작 노드를 제외한 전체 n - 1개의 노드에 대해 반복
 for i in range(n - 1):
 # 현재 최단 거리가 가장 짧은 노드를 꺼내서, 방문 처리
 now = get_smallest_node()
 visited[now] = True
 # 현재 노드와 연결된 다른 노드를 확인
 for j in graph[now]:
 cost = distance[now] + j[1]
 # 현재 노드를 거쳐서 다른 노드로 이동하는 거리가 더 짧은 경우
 if cost < distance[j[0]]:
 distance[j[0]] = cost # 거리 갱신

dijkstra(start)

모든 노드로 가기 위한 최단 거리를 출력
for i in range(1, n + 1):
 # 도달할 수 없는 경우, 무한(INFINITY)이라고 출력
 if distance[i] == INF:
 print("INFINITY")
 else:
 print(distance[i])
```

```
도달할 수 있는 경우 거리를 출력
else:
 print(distance[i])
```

우선순위큐 - 구현은 Minheap, Maxheap

구현방식/시간	삽입시간	삭제시간
리스트	$O(1)$	$O(N)$
힙	$O(\log N)$	$O(\log N)$

- 방문하지 않은 노드 중 거리가 가장 짧은 것을 선택하기 위해 힙을 사용한다.

```
import heapq # 기본은 Minheap
import sys
input = sys.stdin.readline
INF = int(1e9) # 무한을 의미하는 값으로 10억을 설정

노드의 개수, 간선의 개수를 입력받기
n, m = map(int, input().split())
시작 노드 번호를 입력받기
start = int(input())
각 노드에 연결되어 있는 노드에 대한 정보를 담는 리스트를 만들기
graph = [[] for i in range(n + 1)]
최단 거리 테이블을 모두 무한으로 초기화
distance = [INF] * (n + 1)

모든 간선 정보를 입력받기
for _ in range(m):
 a, b, c = map(int, input().split())
 # a번 노드에서 b번 노드로 가는 비용이 c라는 의미
 graph[a].append((b, c))

def dijkstra(start):
 q = []
 # 시작 노드로 가기 위한 최단 경로는 0으로 설정하여, 큐에 삽입
 heapq.heappush(q, (0, start))
 distance[start] = 0
 while q: # 큐가 비어있지 않다면
 # 가장 최단 거리가 짧은 노드에 대한 정보 꺼내기
 dist, now = heapq.heappop(q)
 # 현재 노드가 이미 처리된 적이 있는 노드라면 무시
 if distance[now] < dist:
 continue
 # 현재 노드와 연결된 다른 인접한 노드들을 확인
 for i in graph[now]:
 cost = dist + i[1]
 # 현재 노드를 거쳐서, 다른 노드로 이동하는 거리가 더 짧은 경우
 if cost < distance[i[0]]:
 distance[i[0]] = cost
 heapq.heappush(q, (cost, i[0]))
```

```

다익스트라 알고리즘을 수행
dijkstra(start)

모든 노드로 가기 위한 최단 거리를 출력
for i in range(1, n + 1):
 # 도달할 수 없는 경우, 무한(INFINITY)이라고 출력
 if distance[i] == INF:
 print("INFINITY")
 # 도달할 수 있는 경우 거리를 출력
 else:
 print(distance[i])

```

```

import heapq
INF = int(1e9)
BASE = 10**6

N, M = map(int, input().split())
company = [int(input()) for _ in range(N)] # n번째 역을 운행하는 회사 (0또는1)
tmp = [list(map(int, input().split())) for _ in range(N)] # 인접행렬 (역 간 이동시
간)
graph = [[] * N for _ in range(N)] # 이동한 역 정보
distance = [INF for _ in range(N)] # 최단거리 저장

def dijkstra(start, distance):
 q = []
 heapq.heappush(q, (0, start))
 distance[start] = 0

 while q:
 dist, now = heapq.heappop(q)

 if dist <= distance[now]:
 for pos, c in graph[now]: # 현재 노드 (now)와 연결된 모든 인접 노드 탐색
 # pos는 인접노드번호, c는 현재 노드에서 인접 노드까지의 가중치(이동 시
 간)

 cost = dist + c

 if cost < distance[pos]: # 방금 계산한 거리가 더 짧다면
 distance[pos] = cost # 테이블 갱신
 heapq.heappush(q, (cost, pos))

for i in range(N):
 for j in range(N):
 if tmp[i][j] != 0: # 출발 역i와 도착 역j 회사가 동일
 if company[i] == company[j]:
 graph[i].append((j, tmp[i][j]))
 else: # 환승하는 경우
 graph[i].append((j, tmp[i][j] + BASE)) # BASE 환승 시간 추가

```

```
dijkstra(0, distance)

count, time = distance[M] // BASE, distance[M] % BASE
print(count, time)
```

## 우선순위 큐

**우선순위 큐** 우선 순위가 높은 데이터가 큐에서 먼저 나간다. 구현방법 : 배열, 연결리스트, 힙

1. 배열
2. 연결리스트
3. 힙 **힙** 완전이진트리의 일종으로, 우선순위 큐를 위한 자료구조
  - 최댓값/최솟값을 빠르게 찾을 수 있다.
  - 부모 노드의 키 값이 자식 노드 값보다 항상 크다(작다.)
  - 이진 탐색 트리와 달리 중복된 값을 허용한다.
  - 항상 루트 노드를 먼저 제거한다. <minheap maxheap 사진>

### 구현

- 배열에 저장한다.
- 편의를 위해 첫 번째 인덱스 0은 사용하지 않는다. <강의교안 사진>

**Min-Heapify (Max-Heapify)** : 힙에 새 원소를 삽입했을 때 다시 힙이 되도록 만드는 함수

## sets (서로소 집합, 분리 집합, union find)

교집합이 없는 두 개 이상의 집합

각 집합은 대표원소가 있다.

### operations

**MAKE-SET(x)** x원소를 갖는 집합 {x} 추가  $O(1)$  **UNION(x,y)** x,y원소가 속한 집합 두개를 병합 (대표원소는 1개)  
**FIND-SET(x)** x원소가 속한 집합의 대표 원소 리턴  $O(1)$  union과 find-set 사이에는 tradeoff 관계가 있다.

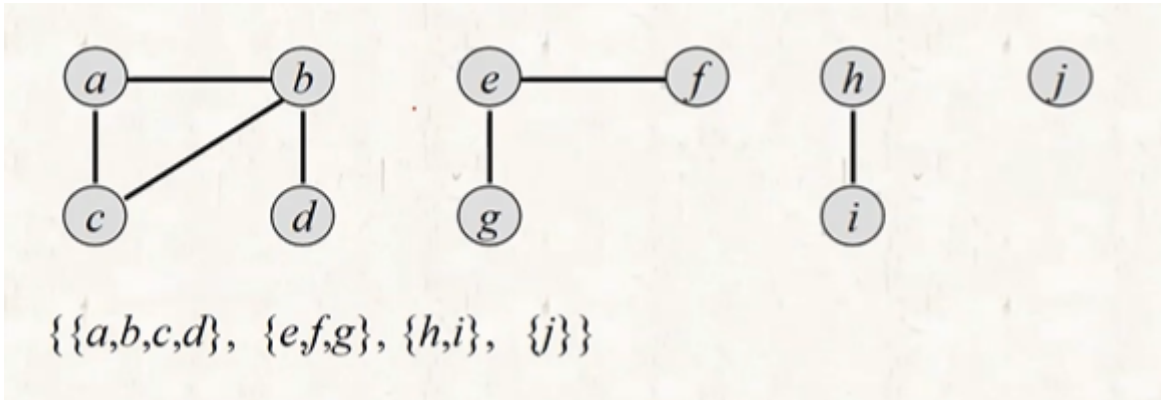
### parameters

- total operations : m
- MAKE-SET operations(전체 원소의 갯수) : n
- UNION operations : u FIND-SET operations : f
- $m = n + u + f$

- $u \leq n-1$ 
  - UNION은 전체 집합의 수  $n$ 를 -1 줄인다. UNION을  $n-1$ 번 하면, 전체가 하나의 집합이 되고 더이상 UNION을 할 수 없다.

## 연결 요소(connected component)

연결 요소 하나는 하나의 집합을 의미한다.



- 연결 요소에 속한 모든 정점을 연결하는 경로가 있어야 한다.
- 또 다른 연결 요소에 속한 정점과 연결하는 경로가 있으면 안된다.

1. static graph DFS 탐색을 통해 구할 수 있다.  $O(V+E)$

2. dynamic graph graph가 변화할 때마다 DFS를 다시 해야하므로 dynamic graph에서는 DFS 방법이 적합하지 않다. disjoint set 자료구조를 사용한다.

```

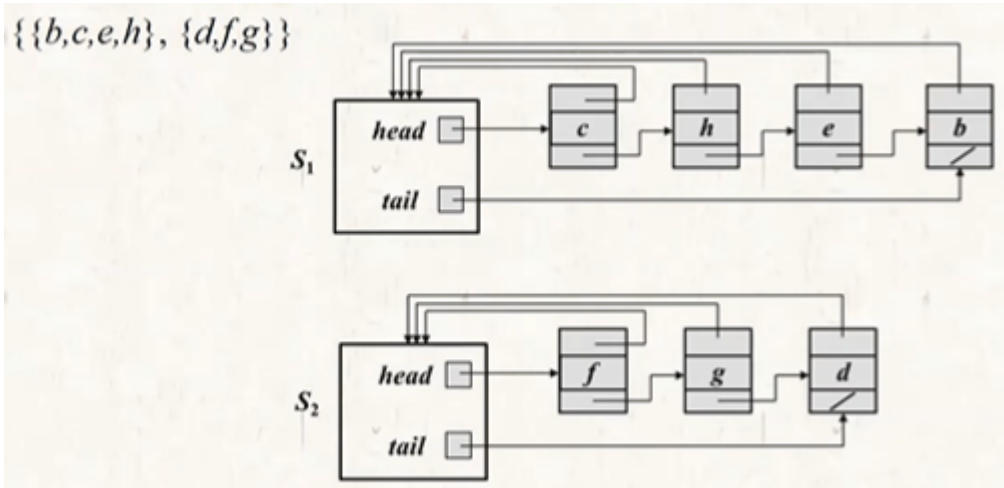
CONNECTED-COMPONENTS(G)
1 for each vertex $v \in G.V$
2 MAKE-SET(v)
3 for each edge $(u, v) \in G.E$
4 if FIND-SET(u) $\neq$ FIND-SET(v)
5 UNION(u, v)

```

이제 서로소 집합의 구현 방법에 대해 알아보자.

### 1. Linked-list

- 각 집합은 하나의 linked list로 표현된다.
- 첫번째 원소가 대표 원소이다.
- 모든 노드가 대표 원소를 가리키는 포인터를 갖는다.
- 집합 내 원소의 순서는 상관 없다.



tail 포인터 : union 할 때 마

지막 원소의 다음을 가리키기 위해 필요

**MAKE-SET(x)**  $O(1)$  **UNION(x,y)**  $O(m)$  -  $m$ 은 더 짧은 리스트의 원소 갯수  $x,y$ 는 각 linked list를 가리키는 포인터.

- weighted union heuristic
  - 더 짧은 list를 뒤에 붙인다.

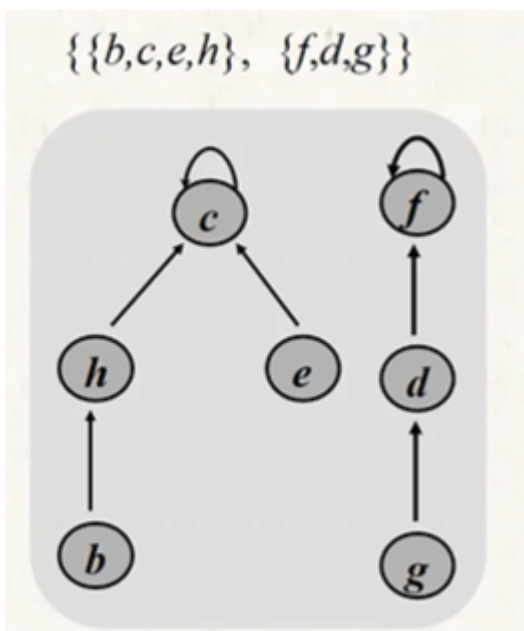
tail pointer, 두 리스트를 연결하는 pointer,

**FIND-SET(x)**  $O(1)$   $x$  포인터가 가리키는 원소가 속한 집합의 대표원소 리턴

$u \cdot n \leq (n-1) \cdot n$   $u$ 번 union할 때 각 노드가 최대  $\log n$ 번 선택됨

## 2.Forest (tree 집합)

- 각 집합을 하나의 tree로 표현
- root가 대표 원소
- 화살표가 위로 향한다!!
- child 간 순서?



**MAKE-SET(x)**  $O()$  **UNION(x,y)**  $O()$  - 더 높은 높이의 tree로 합친다.

- union by rank (rank : 루트에 포함된 tree의 upper bound)

**FIND-SET(x)**  $O(m)$  a(n) : n이 아무리 커져도  $\leq 4$

## DP (동적계획법)

: 문제를 더 작은 크기의 동일한 문제들로 나누어 이 작은 문제들의 답을 이용해 최종 답을 구하는 기법

### <조건>

1. 최적 부분 구조인가?

: 큰 문제를 작은 문제로 나눌 수 있고, 작은 문제의 답을 모아 큰 문제를 해결할 수 있다.

2. 중복되는 부분 문제

: 동일한 작은 문제를 반복적으로 해결해야 한다.

### <문제 접근법>

먼저 완전탐색, 그리디, 구현 등의 방법으로 풀 수 있는지 체크

-> 불가능하다면 DP로 해결

1. dp테이블의 i번째 인덱스를 어떻게 정의할 것인가
2. 부분문제들로 어떻게 나눌지
3. 가장 작은 부분문제가 무엇인지, 이때의 상태는 무엇인지
4. 점화식 세우기

### 1. 하향식 방법

- **메모이제이션**(캐싱)을 이용해 이미 해결한 작은문제의 답을 기록
- **재귀**함수로 구현
- 시간복잡도 :  $O(n)$

```
피보나치 수열을 저장하는 dp배열
d = [0] * 100

def fibo(x):
 # 종료 조건(1 혹은 2일 때 1을 반환)
 if x == 1 or x == 2:
 return 1
 # 이미 계산한 적 있는 문제라면 그대로 반환
 if d[x] != 0:
 return d[x]
 # 아직 계산하지 않은 문제라면 점화식에 따라서 피보나치 결과 반환
 d[x] = fibo(x - 1) + fibo(x - 2)
 return d[x]
```

## 2. 상향식 방법

- dp[0]부터 시작하여 반복문을 통해 dp[n]까지 계산을 누적하여 전체문제를 해결한다.
- 하향식 방법에 비해 재귀를 사용하지 않기 때문에 시간과 메모리 측면에서 효율적이다.

```
d = [0] * 100

첫 번째 피보나치 수와 두 번째 피보나치 수는 1
d[1] = 1
d[2] = 1
n = 99

피보나치 함수(Fibonacci Function) 반복문으로 구현
for i in range(3, n + 1):
 d[i] = d[i - 1] + d[i - 2]

print(d[n])
```

상향식	하향식
for문으로 구현	재귀함수로 구현
작은 문제를 모아 큰 문제 해결	큰 문제를 해결하기 위해 작은 재귀 함수 호출
점화식 필요	점화식 필요
메모이제이션 없음	메모이제이션 필요(dp테이블)

오버헤드를 줄일 수 있다. 일반적으로 성능이 더 좋다.    오버헤드 가능성이 있다.

+)재귀함수에서 함수를 불러오고 하는 과정에서 시간이 걸리기 때문에 재귀함수를 사용하지 않고 반복문으로 처리하는 Bottom-Up 방식에 비해 속도가 조금 느리다.

### 예시)1로 만들기 문제

정수 x가 주어졌을 때. 정수 x에 사용할 수 있는 연산은 다음과 같이 4가지이다.

1. x가 5로 나누어 떨어지면, 5로 나눈다.
2. x가 3으로 나누어 떨어지면, 3으로 나눈다.
3. x가 2로 나누어 떨어지면, 2로 나눈다.
4. x에서 1을 뺀다.



정수  $x$ 가 주어졌을 때, 위 연산들을 적절히 사용해서 값을 1로 만들 때, 연산을 사용하는 횟수의 최솟값을 구하라.

- 26->25->5->1

탐다운 방식 풀이

```
x = int(input())

d = [0] * 1000001

for i in range(2, x + 1):
 # 현재의 수에서 1을 빼는 경우
 d[i] = d[i - 1] + 1
 # 현재의 수가 2로 나누어 떨어지는 경우
 if i % 2 == 0:
 d[i] = min(d[i], d[i // 2] + 1)
 # 현재의 수가 3으로 나누어 떨어지는 경우
 if i % 3 == 0:
 d[i] = min(d[i], d[i // 3] + 1)
 # 현재의 수가 5로 나누어 떨어지는 경우
 if i % 5 == 0:
 d[i] = min(d[i], d[i // 5] + 1)

print(d[x])
```